MAARS: Multi-Rate Attack-Aware Randomized Scheduling for Securing Real-time Systems*

Arkaprava Sain, Sunandan Adhikary, Ipsita Koley, Soumyajit Dey
Dept. of CSE, Indian Institute of Technology Kharagpur, India
{arkapravasain,mesunandan}@kgpian.iitkgp.ac.in,ipsitakoley@iitkgp.ac.in,soumya@cse.iitkgp.ac.in

Abstract

In modern Cyber-Physical Systems (CPSs), most of the safety-critical tasks are executed with a fixed sampling period to ensure deterministic timing behaviour. However, adversaries can exploit this deterministic behaviour of periodic safety-critical tasks to launch schedule-based attacks (SBAs) on them. This paper aims to prevent and minimize the possibility of SBAs by switching between strategically chosen periodicities of control tasks while ensuring that their performance remains unhampered. Thereafter, we present a novel schedule vulnerability analysis methodology to switch between valid task schedules generated at run time. Overall, we introduce a novel *Multi-Rate Attack-Aware Randomized Scheduling (MAARS)* framework for preemptive fixed-priority schedulers that minimize the success rate of timing-inference-based attacks on safety-critical real-time systems. We evaluate the efficacy of MAARS in terms of attack prevention on synthetic task sets and an automotive benchmark task set in a *Hardware-in-the-Loop (HIL)* environment.

CCS Concepts

• **Security and privacy → Embedded systems security**.

Keywords

Schedule-Based Attacks, Multi-Rate Control, CPS Security

ACM Reference Format:
Arkaprava Sain, Sunandan Adhikary, Ipsita Koley, Soumyajit Dey. 2025. MAARS: Multi-Rate Attack-Aware Randomized Scheduling for Securing Real-time Systems. In *ACM/IEEE 16th International Conference on Cyber-Physical Systems (with CPS-IoT Week 2025) (ICCPS '25), May 6–9, 2025, Irvine, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3716550.3722030

1 Introduction

Cyber-physical Real-time systems (RTSs) often comprise multiple processors, each running tasks with different criticality levels [4]. Some of these tasks are safety-critical in nature. A major set of such safety-critical tasks are periodic controllers with hard deadlines. For achieving predictable execution behaviour, fixed-priority-based scheduling algorithms remain popular for embedded control units in CPSs, with higher priority typically being assigned to hard real-time safety-critical tasks to ensure that they meet their deadlines. Also, the deterministic nature of static priority scheduling enables designers to analyse the worst-case response time (WCRT) of every control loop and design delay-aware control strategies for them.

However, the predictability of fixed-priority scheduling exposes the timing information of safety-critical task executions. The attacker first finds out the periodicity of a safety-critical task by observing either its data communication in the network or the changes in physical system states of the controlled plant [5, 19]. By compromising a relatively lower priority task (non-safety-critical) that periodically executes immediately before or after this victim task, the attacker observes its execution timestamps for multiple hyper-periods. Since a static fixed-priority schedule is followed, the execution sequence repeats every hyper-period, which helps the attacker infer possible initial offsets of the safety-critical task and predict their future arrival times. The attacker can manipulate data inside the victim task's shared I/O device buffer within a time window around those inferred future arrival time instances [5]. Such vulnerable time windows are termed as *attack effective window (AEW)*, a quantity which is implementation-dependent and can be measured experimentally by the attacker [5]. Such schedule-based control data override attacks are relevant in operating systems that do not provide memory separation between applications, such as the OSEK/VDX family [1]. Although most modern embedded platforms have specialized memory protection units (MPUs) for isolating vulnerable software components, many specialized OS designed with minimal resources lacks memory separation between applications, making them more susceptible to such attacks. These vulnerabilities and associated attack methods have been studied in [7, 18]. SBAs can be categorised into the following four models depending on the timing relation between the victim's arrival and the attacker task's arrival within AEW [18], i.e., posterior Attack (attacker task executes after victim task), anterior Attack (attacker task executes before the victim task), concurrent attack (attacker task executes while the victim task runs) and, pincer Attack (combines both posterior and anterior attack). A posterior attacker can alter actuation messages computed by the victim control task, an anterior attacker can alter sensor readings to be read by the control task, etc. Since the non-critical attacker tasks are of lower priority, the corresponding jobs are mostly scheduled/completed after higher-priority victim task instances in case of nearby arrival times. This makes posterior attacks more prevalent compared to other attack models in embedded controllers [20]. In this paper, we consider the posterior attack model, analyse the drawbacks of state-of-the-art defence mechanisms, and discuss how the proposed defensive solution outdoes them.

In state of the art, *schedule randomization*-based defence approaches are widely explored. One of the initial works in this area is *TaskShuffler* [23], which randomizes job execution by allowing

*Authors acknowledge CHANAKYA Fellowship grant, AI4ICPS, IIT Kharagpur.

ICCPS '25, Irvine, CA, USA

ACM ISBN 979-8-4007-1498-6/2025/05
https://doi.org/10.1145/3716550.3722030

lower-priority jobs in the ready queue to run before higher-priority jobs. In [13], the authors discuss two *slot-level randomization techniques* to mitigate directed schedule-based attacks. However, randomization without insight into the attacker's strategy may create vulnerable schedules, causing successful attacks [18]. The authors in [6] devise a privacy-preserved strategy that samples a suitable execution offset for critical tasks from a Laplace distribution and switches between them to keep the execution sequences from getting exposed. However, they do not ensure the performance and schedulability criteria of the control tasks while doing so. In another line of work [7], all *untrusted* tasks are blocked within the *AEW* of all safety-critical or *trusted* tasks. However, this may result in missed deadlines for *untrusted* tasks. In this work, we propose a schedule randomization policy that is attack-aware and ensures the schedulability of all tasks in the system while also carefully balancing the trade-offs between performance and security.

One of the crucial pieces of information for any SBA model is the future arrival instances of a critical task, which is derived by finding out its periodicity [5]. If the sampling period is dynamically switched, the attacker will find it difficult to infer the exact future arrival time even after doing schedule analysis. With this motivation, we utilise a multi-rate controller scheduling strategy to ensure security in real-time task schedules by reducing its *inferability*, i.e. identifying start times of victim task instances. As shown in the state-of-the-art research works, dynamic switching between a predefined set of sampling periods, coupled with optimally designed controllers, significantly reduces resource utilization, ensuring performance, compared to conventional fixed sampling rate static scheduling [10]. Existing works on multi-rate switching [2, 21, 22] check whether a *Common Quadratic Lyapunov Function (CQLF)* or a *Multiple Lyapunov function (MLF)* [16] exists among sampling rates in order to guarantee stable switching.

We propose a novel *Multi-Rate Attack-Aware Schedule Randomization (MAARS) Framework* to secure periodic safety-critical control tasks in a uniprocessor platform against schedule-based posterior attacks [18]. To the best of our knowledge, this is the first work that proposes an attack-aware schedule randomization policy for control tasks. The main contributions of this work are as follows:

1. We present a novel approach for judiciously choosing performance-aware sampling rates for periodic safety-critical control tasks such that the *inferability* of its future arrival instances are minimized, thereby effectively thwarting the leakage of critical task information *(such as periodicity, initial task offset, etc.)* while preserving control-theoretic properties stability properties using CQLF based switching rules.

2. We propose a novel methodology that generates a set of schedules offline using slot-level randomization techniques for the chosen set of sampling rates for each control task. We also develop a *runtime schedule selection algorithm* that intelligently deploys schedules from this set by observing the posterior attacks' effect on a control loop such that the vulnerability of the corresponding control task is minimized in that schedule.

3. We experimentally evaluate the proposed *MAARS* framework with several synthetic task sets, highlighting the trade-offs between security and system performance. Additionally, we demonstrate the framework's real-time applicability using an automotive case study conducted in a HIL setup.

2 System Model

2.1 Task and Scheduler Model

Let Γ denote a set of N independent periodic real-time tasks $\Gamma = \{\tau_1, \tau_2, ...\tau_N\}$, scheduled on a uniprocessor system by a static priority scheduler with fixed priority. Each task $\tau_i \in \Gamma$ is assigned a unique priority $pri(\tau_i)$. Each task $\tau_i \in \Gamma$ is characterized by the tuple (e_i, p_i, d_i), where e_i is the worst-case execution time (WCET), p_i is the minimum inter-arrival time/minimum separation between two successive jobs, and d_i is the relative deadline. We assume implicit deadlines for each task, meaning $d_i = p_i$, $\forall i \leq N$. The WCET of the task incorporates jitter and context-switch overheads. In this paper, we consider a mixed-criticality system, i.e., it comprises control tasks of varying criticality levels. The criticality values of the tasks are determined based on the vulnerability of the corresponding software components. In our work, we assume that the criticality of the tasks correlates positively with their priorities. Following the work in [7], we consider all the safety-critical control tasks as trusted and hard to compromise. Whereas other tasks are considered to be untrusted and can be compromised by a schedule-based attacker. Any safety-critical control task from the *trusted* set $\Gamma_t \subseteq \Gamma$ can be a victim to SBA launched by an adversary who has compromised a non-safety-critical task from the *untrusted* set $\Gamma_u \subseteq \Gamma$. Due to the high criticality level, the trusted tasks are mostly assigned with higher priority than the untrusted tasks. Hence, their executions are not delayed by the non-critical tasks as they are promptly scheduled as they arrive by a *fixed-priority preemptive scheduler*. In this work, we consider these trusted safety-critical control tasks to be designed with different sampling rates, which changes their periodicities of execution. We denote the set of periodicities for i^{th} control task as $P_i = \{p1_i, p2_i, \cdots, pn_i\}$. We assume that the task set Γ is schedulable by a fixed-priority preemptive scheduler such that the *worst-case response time (WCRT)* $wcrt_i$ for any task $\tau_i \in \Gamma$ satisfies, $wcrt_i = e_i + \sum_{\tau_j \in hp(\tau_i)} \left\lceil \frac{r_i^k}{p_j} \right\rceil e_j \leq d_i$. Here $r_i^0 = e_i$, $wcrt_i = r_i^{k+1} = r_i^k$ for some $k \geq 0$ and for every $\tau_i \in \Gamma_t$, p_i = minimum periodicity from the set P_i. This criteria ensures that the tasks are *schedulable* for their maximum execution rates, and assuming no precedence constraints among the tasks, *valid schedules* can be generated without any deadline misses.

2.2 Control Task Model

We express the plant model as a *Linear Time-Invariant (LTI)* system having dynamics as follows:

$$\begin{aligned} \dot{x} &= A_c x(t) + B_c u(t),\ y(t) = Cx(t) + v(t) \\ \hat{x}[k+1] &= (A - LC)\hat{x}[k] + Bu[k] + Ly[k] \\ u[k+1] &= K\hat{x}[k+1] \end{aligned} \tag{1}$$

Here, the vectors $x \in \mathbb{R}^n, \hat{x} \in \mathbb{R}^n, y \in \mathbb{R}^m$, and $u \in \mathbb{R}^p$ describe the plant state, the estimated plant state, output and the control input, respectively. The matrices A_c and B_c are continuous time matrices that define the continuous-time state and input-to-state transition matrices, respectively. C is the output transition matrix. At the k^{th} sampling instance, the real-time is $t = kh$, where h is the sampling period. We can, therefore, define the discrete-time matrices corresponding to their continuous-time counterparts as

$A = e^{A_c h}$, $B = \int_0^h e^{A_c t} B_c \cdot dt$. K and L are Linear Quadratic Regulator (LQR) controller gain and Kalman estimator gain, respectively. Each

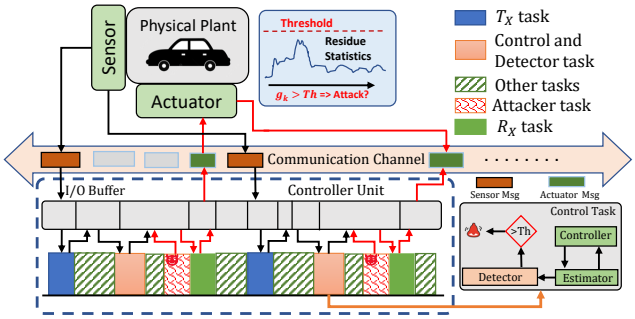


Figure 1: System and Control Task Model

control task samples the sensed plant measurement vector y at each sampling iteration, using which future states of the plant are estimated. Based on the estimated plant state, a control task calculates the required control input u to transmit it to the actuator to actuate and control the plant output. In Fig. 1, we illustrate the operation of a physical plant communicating with a controller unit (depicted within the blue dotted box). The arrows represent data transmission between various tasks scheduled in the controller and the sensor/actuator units through messages in the communication channel. Red arrows indicate instances of data corruption by an attacker task.

2.2.1 Closed Loop Under Sampling Period Variation: As the discrete-time system characteristic matrices change with the choice of h, the LQR control gain K, and Kalman estimator gain L changes. Therefore, the discrete-time matrices A, B, K, L in Eq. 1 can be replaced with their sampling period-dependent versions, i.e., A_h, B_h, K_h, L_h. By discretizing the continuous-time plant equation in Eq. 1 we get $x[k+1] = A_h x[k] + B_h u[k]$. Since we intend to capture the change in the progression of the control loop w.r.t. the change in its sampling period, we define an extended augmented state vector of the control task as $X = [x^T, \hat{x}^T]^T$ that captures both plant and estimator/controller dynamics. By replacing u with $-K_h\hat{x}$ and y with Cx, the overall closed-loop system evolves as follows.

$$X[k+1] = \mathbb{A}_h X[k], \mathbb{A}_h = \begin{bmatrix} A_h & -B_h K_h \\ L_h C & A_h - L_h C - B_h K_h \end{bmatrix} \quad (2)$$

For two different sampling periods $p1$ and $p2$, we denote the discrete-time augmented system matrices by $\mathbb{A}_{p1}$ and $\mathbb{A}_{p2}$. Our methodology uses this augmented system representation in Eq. 2 to ensure a desired performance while varying the periodicities of control tasks for security.

2.2.2 Residue-based Anomaly Detector: Each safety-critical control loop is equipped with a residue-based detector that observes changes in the system residue (i.e., the difference between the sensed and estimated outputs) for anomaly detection. We consider an integrated implementation of controller and detector functionalities as part of the control task to avoid data manipulation. We use a χ^2-based detector in this work. This has been illustrated by the blue box in Fig 1. The χ^2 detector utilizes a normalized quadratic function of the residual to amplify and easily detect minute variations in system residue. For system residue $res[k]$ at k^{th} sampling iteration, its chi-square measure is $z[k] = res[k]^T \Sigma_{res}^{-1} res[k]$ where Σ_{res} is the variance of system residue. Considering the measurement noise to be a zero-mean Gaussian noise, $res[k] \sim \mathcal{N}(0, \Sigma_{res}) \Rightarrow z[k] \sim \chi^2(m, 2m)$, where m is the number of output measurements, considered as the degree of freedom of the χ^2 distribution. We employ a windowed chi-square detector that compares the average value of the chi-square statistic of system residue over a pre-defined time window (N), i.e., $g[k] = \frac{1}{N} \sum_{i=0}^{N-1} z[k-i]$ and compares it with a pre-defined threshold Th. The threshold is calculated to maintain a desired false alarm rate [11]. The chi-squared detector raises the alarm, denoting an attack attempt on a certain closed loop when $g[k] > Th$ at any k^{th} sampling period of that closed loop.

3 Threat Model

In this section, we discuss in-depth how we model the schedule-based attacker and explain our assumptions about the attacker's objectives and capabilities. The objective of the attacker is to utilize the timing information exposed by the processor-level task execution schedules to compromise periodic tasks with higher criticality.

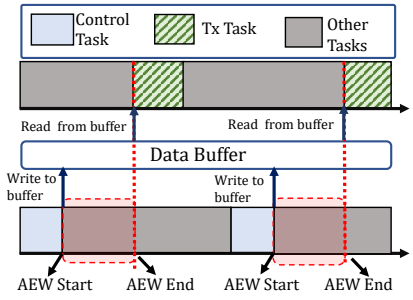


Figure 2: AEW

nondeterministic and difficult to infer. This is why, like prior works in [14, 23], such tasks are not a part of our security consideration. In this work, we assume that critical tasks are implemented using appropriate security primitives (such as cryptographic authentication, integrity monitoring, etc.) to ensure their execution remains 'trusted'. Such implementations, from *trusted* vendors and OEMs, usually go through thorough security and functionality checks, making them difficult to compromise [7, 20]. However, such enclaved execution with security primitives is not done for the low priority non-critical tasks due to the associated resource and timing overheads. Hence, an attacker is more likely to attempt compromising less critical tasks, that are often assigned lower priorities. We denote such compromisable tasks as *untrusted* tasks. Since these untrusted tasks are part of the software that comes from a range of third-party vendors[20], compromising them using a man-in-the-middle-type attack is possible. The following are assumptions regarding the attacker's ability, objectives, and attack methods.

3.0.1 Attacker's Capabilities: (1) It is assumed that the task scheduler cannot be compromised by the attacker. Consequently, the attacker lacks direct control over task execution and is restricted to leveraging compromised tasks only when the scheduler runs them. *(2)* The adversary knows the scheduling policy used, and it can compromise *one task among all the* lower-priority untrusted

tasks (Γ_u). This enables the attacker to observe executions of this compromised untrusted task for several hyper-periods to find out partial information about its intended victim control task($\in \Gamma_t$). *(3)* The attacker can partially deduce the sampling rates of any control task ($\in \Gamma_t$) either by observing the message data packets in the network or physical system states. *(4)* Attacker can modify the data written by the victim control task inside an I/O buffer or cache only if it executes within an *Attack Effective Window (AEW)* [7]. We illustrate this in Fig. 2, where the AEW of the control task has been shaded in red, considering a posterior attacker. AEW starts when the control task (in the blue shade) finishes execution, writes its data to the buffer and ends once the transmission task (in the green stripe) starts. In general, AEW is the specific time interval within which the attacker task must be executed for successful data manipulation. AEW for each control task is dependent on the task schedule and attacker type; it can be derived experimentally [5]. We denote the AEW length for a control task $\tau_i \in \Gamma_t$ by Ω_i.

Tasks	P_i	e_i
τ_1	4	1
τ_2	4	1
τ_3	5	2

Table 1: Task-Set 1

3.0.2 Attack Execution: We assume an attacker's objective is to manipulate control inputs by performing a schedule-based posterior attack on control tasks. To implement such an attack, the attacker must infer the exact future arrival instances of the victim task. Having compromised a lower-priority untrusted task, the attacker can log that task's start and end time as part of the overall schedule. Utilising this information, it attempts to predict the start and end times of other trusted tasks in the schedule. The attacker can achieve this by using a *schedule ladder-based analysis* as proposed in [5]. A schedule ladder represents a task schedule arranged vertically with adjacent timelines of equal row size. Each column in the ladder represents an atomic observation period, or unit time, of real-time duration δ. The attacker considers the row size of the ladder as equal to the victim task's sampling period measured in time units of size δ. Hence, for a given victim task τ_i executing with a sampling rate p_i time units, the row length of the ladder becomes δp_i. The motive behind taking such a row length is that the victim task will arrive once in every row of the ladder, and since each column within the ladder is of δ duration, it occupies δe_i columns in each row of the ladder. Moreover, all arrivals of τ_i will be located in the same column. We illustrate a schedule ladder diagram for the task set in Tab. 1 in Fig. 3. The row length of the ladder is 4 since τ_1(victim) sampling rate = 4. The first row represents the timeline $[0, 4]$, second row $[4, 8]$ and so on. The black dot shows the arrival column of the victim task τ_1 that arrives once in every row and occupies column 1 of the ladder (it can be any column in general). Task τ_2 arrives at the same time as τ_1 (green square) but executes in column 2 based on priority. While performing an analysis, the starting point of analysis (observation) of the top row can be arbitrarily chosen by the attacker, which is decided by the start of its observation window. The attacker starts observing its own start and end times using the compromised untrusted task. Note the attacker task is a lower-priority untrusted task (w.r.t victim). Hence, if the victim task arrives in the same column, it preempts the attacker task. This is shown in Fig. 3, where the attacker task τ_3 arrives (red arrow) at instances $t = 0, 20$ in the first column (since its sampling rate is 5) but gets preempted by τ_1. In column 2, it gets preempted by τ_2. Therefore, it always executes in the third and fourth columns of the ladder.

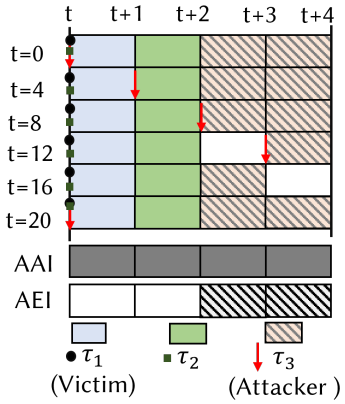


Figure 3: Ladder Diagram of Task-Set-1

After the end of observation, the attacker finds out *Attack Arrival Instances (AAI)*. The AAI is the set of ladder columns where the attacker task has arrived at least once during its entire observation window. For e.g. in Fig. 3, the AAI is $[0, 4]$, shown below the ladder diagram. The set of ladder columns where the attacker task has been executed at least once during its entire observation window are termed as *Attack Execution Instances (AEI)*. Note that the ladder columns where the attacker task arrives ($\in AAI$) but never gets executed ($AAI \setminus AEI$ since it is preempted by higher priority victim tasks) are the possible candidates for the columns where the victim task arrives. Hence, the attacker easily infers these ladder columns, which are in AAIs but not AEIs, as the victim task's arrival column (offset), predicting them as the victim's future arrival instances. We illustrated this in Fig. 3 for task-set Tab. 1, where using τ_3 as an attacker task results in $|AAI| = 4$ and $|AEI| = 2$. One immediate conclusion we draw from this discussion is that a longer execution time for the attacker task makes it easier for an attacker to infer the arrival column of the victim task. This is because the attacker task execution will span a greater proportion of ladder columns, making it easier to guess the correct victim task arrival column.

Consider the task set in Tab. 2, where the trusted control task set $\Gamma_t = \{\tau_1, \tau_2\}$ and the untrusted task set $\Gamma_u = \{\tau_3\}$ (Fig.4). After determining the arrival time of the victim task τ_1, the attacker uses untrusted task τ_3 to launch a posterior SBA on τ_1. We assume the AEW of τ_1 is $\Omega_1 = 1$ time unit (highlighted in the shaded region of the timeframe). Therefore, for a successful SBA, τ_3 must execute within this timeframe after τ_1 finishes executing. Using this threat model, we demonstrate the shortcomings of state-of-the-art secure scheduling policies in terms of mitigating such attacks and balancing performance and security.

Tasks	P_i	e_i
τ_1	{2,3}	1
τ_2	4	1
τ_3	4	1

Table 2: Task Set-2

4 Motivating Example

Let us consider the task set in Tab. 2, comprising trusted control tasks $\Gamma_t = \{\tau_1, \tau_2\}$ and the untrusted tasks $\Gamma_u = \{\tau_3\}$. The control task τ_1 executes at two different sampling rates 2 and 3 time units (δ time unit= $10ms$), respectively. We assume the attacker has compromised the untrusted task τ_3 to launch a posterior attack on the victim task τ_1. The AEW duration for τ_1 is $\Omega_1 = 1$ time unit.
Protection window-based approaches such as in [7] propose restricting the execution of untrusted tasks' job instances within the AEW

of all the trusted tasks. Fig. 4 demonstrates how a task schedule generated following [7] (for the task set in Tab. 2) leads to deadline misses of untrusted tasks and wastage of CPU resources. The first job of τ_1 executes at $[0, 1]$. Next, in $[1, 2]$, the job instance of τ_2 executes. In $[2, 3]$, second job of task τ_1 arrives and executes. However, the job instance of τ_3 will not be allowed to execute at $[3, 4]$ and hence will miss its deadline, even though the CPU remains idle. This leads to inefficient CPU utilisation.

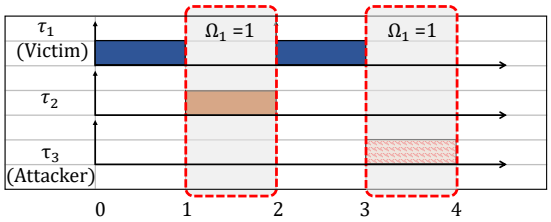


Figure 4: Scheduling of Taskset-2

However, schedule randomization methods are more suitable since they make sure every task meets its deadlines (irrespective of trust). However, the state-of-the-art randomization policies do not consider the attacker's model into consideration and, hence, are often more vulnerable to SBAs while randomly changing the schedules. Earlier works [18] have demonstrated that using attack-unaware randomization policies can still lead to successful attacks. For example, if we apply such a schedule randomization method [23] on the same task set for a single sampling rate of $p_1 = 2$, for control task τ_1, we can generate a total of 8 unique

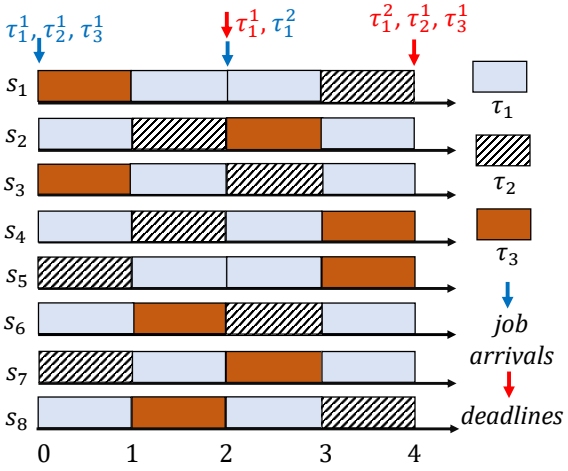


Figure 5: Feasible Schedules Generated for Task-Set 2 ($p_1 = 2$)

feasible task schedules of hyper-period length of 4 time units (see Fig. 5). But among them, 4 schedules s_4, s_5, s_6 and s_7 are vulnerable for τ_1, since instances of τ_3 appears within the AEW of the victim task τ_1. However, if we use two different sampling periods for the control task τ_1, $P_1 = \{2, 3\}$, we can form two possible task specifications Γ_1 and Γ_2 with unique sets of the sampling period, i.e., $\{p_1 = 2, p_2 = 4, p_3 = 4\}$ and $\{p_1 = 3, p_2 = 4, p_3 = 4\}$. For the task specification Γ_2, we can generate 36 unique and feasible schedules (using TaskShuffler) with a hyper-period length of 12 time units. By analysing the vulnerability of τ_1 for all 36 schedules, 12 schedules are found to be safe (no posterior attackable instances within AEW) for victim task τ_1. Thus, multi-rate execution enables the generation of a higher number of randomized schedules with minimal vulnerability. From these examples, we conclude that utilising multi-rate for trusted tasks along with imparting attack awareness to the task schedules (w.r.t trusted tasks) can hamper a schedule-based posterior attacker's success rate.

5 Methodology

In this section, we present the methodology for generating multi-rate attack-aware randomized schedules. A step-wise overview of

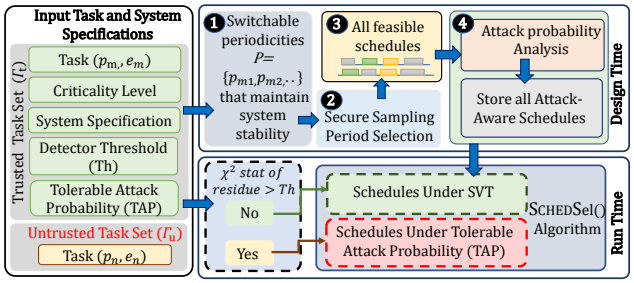


Figure 6: Overview of Proposed MAARS Framework

the MAARS methodology is shown in Fig. 6. During the design time, our proposed framework takes the following as inputs: **(i)** a task set Γ consisting of both trusted and untrusted set of tasks, $\Gamma_t, \Gamma_u (\subseteq \Gamma)$ respectively, **(ii)** trusted task specifications, i.e., criticality level c_i, a *controllable, schedulable* set of periodicities P_i, WCRT e_i of each trusted task $\tau_i \in \Gamma_t$, **(iii)** physical system specifications corresponding to each control task $\tau_i \in \Gamma_t$, its performance specification, **(iv)** other tunable design parameters related to MAARS, like *Tolerable Attack Probability*, *Schedule Vulnerability Threshold*, etc. to constrain the vulnerability of MAARS as defined later in Sec. 5.3. As seen from Sec. 4, switching between the allowable set of sampling periods P_i reduces the determinism (explained as *inferability* in Sec. 5.2) in the real-time task execution pattern. The *first* step in our methodology is to prune a set of periodicities P_i for a given critical/trusted task such that the system, in closed loop with this control task, delivers its desired performance (see Fig. 6). In the *second* step, we intelligently select a subset of secure sampling periods from this performance-aware set of periodicities, such that the attacker's schedule analysis is maximally hampered. We theoretically prove that the devised secure periodicity selection policy maximizes the non-determinism in the execution pattern of the victim control task by preventing the attacker from inferring critical task parameters. The *third* step uses these performance-aware and secure set of sampling rates to generate all possible valid task schedules by following the schedule randomization strategy given in [23]. These *valid* schedules are generated by ensuring that no task misses their deadlines while switching between the periodicities. In the *fourth* step, we analyse the probability of posterior schedule-based attack (SBA) on each victim task for each schedule. By quantifying the vulnerability of each schedule, we rank and suitably store the schedules for using them in runtime to prevent posterior SBAs.

During run-time, MAARS deploys a secure schedule selection algorithm. As discussed earlier, each control task is implemented with a windowed χ^2-detector that flags any attack attempt on its corresponding closed-loop (*box with black-dotted outline* in Fig. 6, refer to Sec. 2.2.2). Under no attack, MAARS randomly selects and deploys different schedules from a designer-selected set of valid schedules with minimum vulnerability. We term this mode as *Normal Mode* (*green-dotted outlined* box in Fig. 6). MAARS switches to an *Alert Mode* when the detector flags an attack on a certain victim task $\tau_i \in \Gamma_t$(*red-outlined* box in Fig. 6). In this mode, the schedule selection algorithm switches to the least vulnerable schedule for victim task τ_i to ensure minimum posterior attack success. Next, we discuss each design time step and run-time algorithm in detail.

5.1 Performance-aware Period Selection

While multi-rate scheduling of control tasks may lead to reduction in the *inferability* of victim task parameters, changes in sampling rate of control loops also means switching between different subsystems created due to such rate changes. In such cases, to ensure stability and desired performance, we must either constrain the switching choices or judiciously choose the rates allowed. In this work, we do the latter. Let us consider, for the augmented closed loop system (see Eq. 2) corresponding to a control task $\tau_i \in \Gamma_t$, this sampling period switching signal is $\sigma_i : \mathbb{N} \mapsto P_i$, where $P_i = \{p1_i, p2_i, \cdots, pn_i\}$ represents the chosen set of non-zero sampling periods. In the presence of this switching signal, we can express Eq. 2 for the control task τ_i as shown below.

$$X^{(i)}(k+1) = \mathbb{A}^{(i)}_{\sigma_i(k)}(X^{(i)}(k)) \tag{3}$$

The switching signal $\sigma_i(k) = pj$ signifies that at k^{th} sampling instance, this augmented system switches to the controller designed with sampling period pj. Now, the following condition (from [16]) must be satisfied for maintaining a desired performance while arbitrarily switching between subsystems (i.e. $\mathbb{A}^{(i)}_{\sigma_i(k)}$-s).

Claim 1. *In the case of a switched system like Eq. 3, in order to maintain a desired* global uniform exponential stability (GUES) *decay rate of γ while arbitrarily switching among different sub-systems, there must exist a common Lyapunov function (CLF) for all its arbitrarily switchable subsystems that satisfy the following set of equations*

$$\kappa_1(\|X^{(i)}[k]\|) \leq V_i(X^{(i)}[k]) \leq \kappa_2(\|X^{(i)}[k]\|) \tag{4}$$
$$\triangle V_i(X^{(i)}[k]) \leq \alpha_i V_i((X^{(i)}[k]), \textit{for } \alpha_i = g(\mathbb{A}^{(i)}_{pj}) \textit{s.t. } \sigma_i(k) = pj$$

Here, κ_1,κ_2 are class $\mathcal{K}_\infty$ functions, and g:$\{\mathbb{A}^{(i)}_{p1}, \cdots, \mathbb{A}^{(i)}_{pn}\} \mapsto \mathbb{R}\backslash\{0\}$. *Global uniform exponential stability (GUES)* with a decay rate $\gamma < 0$ signifies that at k^{th} sampling iteration $\|X(k)\| \leq Me^{\gamma k}\ \|x[0]\|$, where $\|.\|$ is vector norm and $M > 0$. We consider a common quadratic Lyapunov function (CQLF) $V_i(X^{(i)}) = X^{(i)}\mathcal{P}^{(i)}X^{(i)^T}$. We validate the existence of this CQLF for all switchable closed loops by solving the following *linear matrix inequalities (LMIs)* and finding out a positive definite matrix $\mathcal{P}^{(i)} > 0$ such that for given α_j, $\mathbb{A}^{(i)^T}_{pj}\mathcal{P}^{(i)}\mathbb{A}^{(i)}_{pj} - \mathcal{P}^{(i)} \leq \alpha_j\mathcal{P}^{(i)}\ \forall pj \in P_i$. Given a large set of periodicities, we only consider the set of sampling periods for which their corresponding closed loops have a CQLF that follows Claim 1 for a given performance criteria. This ensures that desired performance is guaranteed even under arbitrary switching among the choice of periods for the control loops.

5.2 Period Selection and Schedule Generation

Following the previous step, which ensured control performance, we further prune the set of performance-aware sampling periodicities to maximally reduce the leakage of critical task parameters of control tasks (e.g., arrival times, execution offsets). One may note that under the following two conditions, the ability of an attacker to infer more accurately the correct future arrival instances of victim increases (refer Fig. 3). If the attacker chooses a victim task with a smaller periodicity to construct a schedule ladder, the ladder will have smaller row lengths, and the number of columns not covered inside attacker *AAI* and *AEI* will be less. Also note that an attacker task will prefer having a long execution time as it increases the chance of getting preempted by a victim instance and, thereby, increasing the cardinality of *AEI*.

We defend against such an attacker by suitably choosing a set of sampling periods for the higher-priority victim tasks so that the chances of preemption are reduced (recall that attacker infers timing information of victim task when it is preempted by the victim). To quantify the *inferability* of victim task arrivals, considering an attacker task in a schedule, we introduce a metric *inferability ratio* that represents the number of preemptions by the victim task.

Definition 1. *(Inferability Ratio) The Inferability Ratio (IR) of a schedule s for a victim task, τ_i having minimum sampling rate p_i and an attacker task τ_j, is defined as the ratio between the total number of columns in the schedule ladder where the attacker task τ_j arrives and gets executed and the total number of columns where the attacker task τ_j arrives. Mathematically, Inferability Ratio IR = $\frac{|AEI|\%|AAI|}{|AAI|}$. The modulo operation ensures IR=0 when $|AAI| = |AEI|$, i.e. the attacker task executes in all the columns it arrives at, indicating that there were no preemptions by the victim task.*

If an attacker task is executed in *more* number of columns in a schedule ladder designed for a victim task, the IR of that schedule, w.r.t. the victim, and its attacker choice is *high*. This limits the number of columns where the attacker task arrives but does not get executed due to preemption by higher-priority victim tasks. Therefore, it is easier for the attacker to guess the arrival column of the victim task. Intelligent selection of sampling rates can reduce the IR by reducing the number of columns where the attacker task is preempted by the victim task, i.e., the attacker arrives but does not execute. We claim the following to ensure reduced preemption of the attacker task by suitable selection of victim periodicities.

Claim 2. *Consider an attacker task $\tau_j \in \Gamma_u$ and a victim control task $\tau_i \in \Gamma_t$. For τ_i, let the set of periodicities allowed be P_i with minimum sampling period $p_i = \min\{p \mid p \in P_i\}$. In this scenario, the generated fixed-priority preemptive schedule shall have the least Inferability Ratio (IR) for victim τ_i if for each $p' \in P_i \setminus \{p_i\}$: $p' = np_i + e_j, n \in \mathbb{N}^+$ for most practical cases.*

Proof of Claim 2 is given in the Appendix section. The above periodicity section ensures minimal overlap of the two recurrent task pairs. However, this periodicity selection criteria used in Claim 2 heavily restricts the periodicity choices since higher values of

$n \in \mathbb{N}^+$ are often not supported by Claim 1. We update the criteria as follows in order to get multiple periodicity choices for a victim task with reduced IR.

Remark 1. *For a victim task τ_i, let the set of periodicities allowed be P_i with minimum sampling period $p_i = \min\{p \mid p \in P_i\}$. In this scenario, the generated fixed-priority preemptive schedules shall have a* reduced *Inferability Ratio if for each $p' \in P_i \setminus \{p_i\} : p' = np_i + k'$, $n \in \mathbb{N}^+$, such that $k' \in [e_j, p_i - 1]$ considering an attacker task τ_j with periodicity p_j and execution time e_j. Since $k' \geq e_j$, it ensures zero preemption against any attacker, i.e., IR = 0. Since, $k \leq (p_i - 1)$, the predicted preemption after every $LCM(p_i, p_j)$ time is not repeated and delayed. This clearly reduces the cardinality of AEI, ensuring the reduction in IR compared to the situation when the victim task was scheduled with p_i.*

We prune the performance-aware set of periodicities P_i to make it security-aware. For each victim task τ_i, we eliminate the periodicities from the set P_i that do not satisfy the criteria given in Remark 1. Once all the possible sets of P_i for each victim control task τ_i are pruned, we utilise *TaskShuffler* [23], a schedule randomization technique. Note that after previous steps our task set $\Gamma = \Gamma_t \cup \Gamma_u$, has a set of period choices for tasks in Γ_t. Each control task $\tau_i \in \Gamma_t$ is assigned a performance-aware and secure set of sampling periodicities, i.e. $P_i = \{p_i^1, p_i^2 ..., p_i^{n_1}\}$. For $|\Gamma_t| = q$, the number of task specifications for Γ is given by: $|P_1| \times \cdots \times |P_q|$, as the periods of tasks $\in \Gamma_u$ are assumed unique. Each task specification is given as input to the *TaskShuffler* algorithm for generating the possible set of randomized as well as feasible schedules where no job instance misses the deadline. We denote the overall set of randomized schedules generated considering all task-specifications by $\mathcal{S}$.

5.3 Schedule Vulnerability Analysis

In order to characterize each *valid*/feasible fixed-priority preemptive schedule's vulnerability level, we first formalise the task schedule s as an array, $s = \{s[1], s[2], \cdots, s[l]\}, \forall\, s[j] \in \{0, 1, \cdots, N\}$. Here, l is the number of time units (each time unit of size δ, same as the column sizes of the schedule ladder (refer to "Attack Execution") in the schedule hyper-period (the time duration after which a static task schedule repeats itself). The element $s[j]$ in the array s denotes the task with index $s[j]$ executing at the j-th time unit i.e., $s[j] = i \Rightarrow \tau_i \in \Gamma$ is executed at δj time. We use 0 to denote an idle period in the schedule, and $i \in \{0, 1, ...N\}$ denotes the index of the executing task. For example, $s = \{1, 2, 1, 3\}$ refers to a fixed-priority preemptive schedule for a task set 2. Job 1 of τ_1 executes first in the execution order, followed by job 1 of τ_2 and so on.

Given any schedule $s \in \mathcal{S}$, executing multiple trusted and untrusted tasks from the task set Γ, first we count the total number of possible posterior attacks on s. Given a victim task τ_i with an AEW length of Ω_i, Eq.5 gives a total count of all possible posterior attacks on task τ_i denoted by $C_p(\tau_i)$ in a schedule s of length l.

$$C_p(\tau_i) = \sum_{j=1}^{l} I\left((s[j] == i) \wedge \bigvee_{\forall k \in [1, \Omega_i]} \exists \tau_p \in \Gamma_u, (s[j+k] == p) \right) \tag{5}$$

Here, $I(x)$ is an indicator function defined as follows: $I(x) = 1$ if x is true, else $I(x) = 0$ for a proposition x that returns true/false. The indicator function returns 1 if any job from $\tau_p \in \Gamma_u$ executes in any of the AEW Ω_i of task $\tau_i \in \Gamma_t$. Even though the attacker can compromise a single untrusted task, the system designer analyses the vulnerability of a posterior attack on each trusted task $\tau_i \in \Gamma_t$ by assuming every untrusted task $\tau_j \in \Gamma_u$ as a potential attacker.

Since sampling rates of all the control tasks $\tau_i \in \Gamma_u$ across different schedules vary, the number of job executions of the task that arrive in each schedule hyper-period will also vary. Therefore, it is necessary to express this posterior attack counts on a trusted task τ_i for a schedule $s_k \in \mathcal{S}$ in probabilistic terms. We denote this as the *Attack Probability*, which is defined as follows:

Definition 2. *(Attack Probability) Given a valid schedule $s_k (\in \mathcal{S})$ generated for a task set Γ, the Attack Probability $AP_{<\tau_i, s_k>}$ of a victim task $\tau_i \in \Gamma_t \subseteq \Gamma$, which is executing with a sampling period p_i, is the ratio between the total number of posterior attack counts $C_p(\tau_i)$ and the total number of job-arrivals of the task τ_i in the schedule hyper-period of length l_k. Mathematically, $AP_{<\tau_i, s_k>} = \frac{C_p(\tau_i) \times p_i}{l_k}$.*

For each trusted control task $\tau_i \in \Gamma_t$, the $AP_{<\tau_i, s_k>}$ is different in a schedule s_k. We impose a threshold on the attack probability (AP) to determine which schedules are permissible. We call this Tolerable Attack Probability (TAP). Several existing studies [3, 12] have focused on determining a minimum length *stealthy* false data injection (FDI) attack for a given safety-critical control loop. This essentially means the minimum consecutive number of attack instances that must be injected into the system to cause instability or unsafe behaviour while avoiding detection by existing state-of-the-art intrusion detection mechanisms.

Definition 3. *(Tolerable Attack Probability) Given a safety-critical control task $\tau_i \in \Gamma_t$, let the minimum length for stealthy but successful attack be κ. In a valid schedule s_k with hyper period length l_k unit, if κ instances of τ_i occur in n consecutive hyperperiods, then* Tolerable Attack Probability *for τ_i in schedule s_k is $TAP_{\langle \tau_i, s_k \rangle} = \frac{(\kappa - 1) \times p_i}{n \times l_k}$.*

For a control task τ_i, if the schedule is obvious from context, we denote its TAP by TAP_i. In a mixed-criticality system, the system designer chooses the criticality values of the tasks based on their importance to the system's operations and how their compromise would affect the system's overall performance [15]. Considering tasks with a higher index have a lower priority value, the q safety-critical tasks $\{\tau_1, \tau_2, .., \tau_q\}$ are arranged in decreasing order of priorities. We assign them with their criticality values $\{q, (q-1), \cdots, 2, 1\}$ respectively. We normalize the criticality values by their total sum of criticality values to derive a *criticality level* of each task. Therefore, we denote the normalized *criticality levels* for a task τ_i as $cl_i = c_i / \sum_{j=1}^{k} c_j,\ \forall \tau_j \in \Gamma_t$. Since AP is task-specific, a schedule may have a minimum AP associated with one safety-critical task, whereas it might have very high APs for some other safety-critical tasks. To secure the schedule w.r.t. all victim tasks, we use these criticality levels to decide the APs of which higher criticality tasks should be given more importance while quantifying the level of vulnerability of the overall task execution schedule. We define a *Schedule Vulnerability Index (SVI)* for each schedule that is expressed as a weighted (with criticality levels) average of the APs of all high-criticality tasks.

Definition 4. *(Schedule Vulnerability Index) Given a schedule $s_k \in \mathcal{S}$ generated by a task set Γ, its* Schedule Vulnerability Index, *denoted by SVI_k, is the weighted sum of attack probability $AP_{<\tau_i,s_k>}$, with normalized task criticality levels c_i for each control task τ_i. Mathematically, $SVI_k = \sum_{\forall \tau \in \Gamma_t} AP_{<\tau_i,s_k>} \times cl_i \ \forall \tau_j \in \Gamma_t$.*

Similar to TAP, we intend to find a tolerable upper limit of SVI. We term this as *Schedule Vulnerability Threshold* (SVT). The schedules having SVI above SVT are not *normally* (we shall define what we mean by normally in the next section) deployed in order to reduce vulnerability against posterior SBAs. This ensures that the attack probability thresholds of the higher-criticality (here same as priority) trusted tasks are taken more into consideration during this vulnerable schedule elimination process than that of the lower-criticality victim tasks. Therefore, the *Schedule Vulnerability Threshold* denoted by SVT can be determined by replacing the Attack Probability with TAP as $SVT_k = \sum_{i=1}^{q} TAP_i \times cl_i$. The attack-aware schedules generated using the methodology mentioned in Sec. 5.2 are pre-computed and stored along with their SVIs and victim task-wise AP values after this vulnerability analysis (See Fig. 6). This process eliminates the need for complex online computations in real-time, thereby minimizing the runtime overhead associated with schedule randomization. All valid fixed-priority schedules are stored in an array $\mathcal{A}$, sorted in the order of their respective SVIs. At the K-th index, the schedule with SVT value is stored. Another data structure, *Schedule Lookup Table* (denoted with LUT), stores task-wise an array of schedule indices (from $\mathcal{A}$) which show less *attack probabilities* w.r.t. a victim task than its TAP. An array of schedule indices stored in the j-th row of LUT is sorted in the increasing order of AP w.r.t. task τ_j. For example, if the i_1, i_2-th schedules from $\mathcal{A}$ (i.e., $\mathcal{A}[i_1] = s_{i_1}$, $\mathcal{A}[i_2] = s_{i_2}$) are stored at k, k'-th positions of $LUT[j]$ (j-th row in LUT) respectively, with $k < k'$, then $AP_{\langle \tau_j, s_{i_1} \rangle} < AP_{<\tau_j,s_{i_2}>} \leq TAP_j$.

5.4 Schedule Selection Algorithm

The runtime algorithm in the MAARS framework is responsible for randomly deploying schedules in an attack-aware fashion. The runtime schedule selection function SchedSel() in Algo. 5.4 takes the following inputs, **(i)** The lookup table LUT and the stored schedule array $\mathcal{A}$, **(ii)** the χ^2-detector flag $Atkflag$, **(iii)** the schedule vulnerability threshold SVT and **(iv)** the schedule deployed in the current hyper-period s_k. The algorithm starts by storing the schedule index in $\mathcal{A}$, which has the same SVI as the input SVT. We use the IndexOfSVI() method, which takes the sorted schedule array $\mathcal{A}$ and the SVT as inputs to find out this index and store in K (see line 2). In line 3, by using IndexOf() method we find out the index k of the input schedule s_k that is currently deployed. Another variable, k', is also initialised (with -1) to store the index of the next schedule. The algorithm has two distinct modes of schedule selection and deployment operations depending on the detector flag $Atkflag$. $Atkflag$ is raised if the χ^2 detector exceeds the threshold at any point during the previous hyperperiod.

(1) Normal mode: When there are no attack flags raised, i.e., $Atkflag = 0$ denoting no attack on any control task, the algorithm operates in *Normal Mode* (lines 5 to 7). In this mode, schedules with SVI less than SVT are randomly selected from the array $\mathcal{A}$. Line 4 of the algorithm checks if the $AtkFlag$ value is zero. If it is zero, then the algorithm selects a random number in the range $[0, K-1]$ and stores it in k' (see line 6). Since MAARS randomly deploys new non-vulnerable schedules at every hyper-period, the random schedule index k' must not match with the currently deployed schedule index k. The algorithm keeps picking another random number until $k \neq k'$ for this purpose (see the while loop in line 5). The next schedule $s_{k'}$ is selected from the k'-th row of the schedule array $\mathcal{A}$ (see line 7).

Require: Schedule Array $\mathcal{A}$, Schedule vulnerability threshold SVT, Lookup table LUT, Detector flag $Atkflag$, current schedule s_k
Ensure: next attack-aware schedule $s_{k'}$
1: **function** SchedSel($\mathcal{A}$, SVT, LUT, $Atkflag$, s_k) ▷ Function
2: $K \leftarrow$ IndexOfSVI($\mathcal{A}, SVT$) ▷ Storing schedule index with SVI=SVT
3: $k \leftarrow$ IndexOf(s_k), $k' \leftarrow -1$ ▷ current and next schedule indices init
4: **if** $AtkFlag == 0$ **then**
5: **while** $k' \neq k$ **do**
6: $k' \leftarrow rand()\%K$ ▷ random index between [0,K)
7: $s' \leftarrow \mathcal{A}[k']$ ▷ Normal Mode
8: **else if** $AtkFlag > 0$ **then** $i \leftarrow AtkFlag$ ▷ If attack detected on τ_i
9: **while** $k \neq k'$ **do**
10: $M \leftarrow$ LengthOf($LUT[i]$) ▷ schedule count with AP below TAP_i
11: $ki \leftarrow rand()\%M$, $k' \leftarrow LUT[i][ki]$ ▷ random index ∈[0,M-1]
12: $s_{k'} \leftarrow \mathcal{A}[LUT[i][k']]$ ▷ Alert Mode
13: **return** $s_{k'}$;

Algorithm 1: Runtime Schedule Selection Algorithm

(2) Alert Mode: If the χ^2*-detector* running inside the control task raises an alarm by detecting an attack ($g[k] > Th \Rightarrow AtkFlag > 0$, see Sec. 2.2.2) on a control task τ_i, the algorithm runs in *Alert mode*. In this mode, schedules are selected from the schedule array $\mathcal{A}$ such that the attack probability for the victim control task τ_i is lower than its tolerable attack probability TAP_i. In line 8, if $AtkFlag$ is non-zero, we first store its value in i. The value of the $Atkflg$ denotes a potential attack on task τ_i. In line 10, we find out the length of the i-th row in LUT and store it in M. This is used to find a random number ki in the range $[0, M-1]$ (see line 11). This number is then used to get the index of the next schedule k' from the ki-th column and i-th row of LUT. Note that each such schedule index picked from LUT satisfies $AP_{<\tau_i,\mathcal{A}'_k>} < TAP_i$. As stated earlier, this random number selection is repeated if the index of the current schedule k matches with the randomly picked next schedule index k' (see the while loop in line 9). We select the schedule $s_{k'}$ from the $k'(\neq k)$-th row of $\mathcal{A}$ (see line 12). Finally, SchedSel() returns the new schedule s'_k for deployment. Algo 5.4 is run once every hyper-period in order to select and deploy new schedules randomly. This online schedule selection is done in an attack-informed way. As a result, the vulnerability level of the task execution schedule is minimized while maintaining the system's performance.

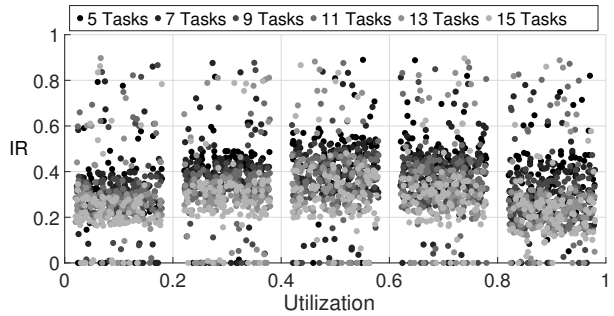
Overhead Analysis: Since we store the schedules in the sorted orders of SVI and AP in arrays, the Algo. 5.4 can access them in $O(1)$. For this a total memory of $M\left[q \cdot \lceil \log_2 M/8 \rceil + L \cdot \lceil \log_2 L/8 \rceil\right]$ bytes is required for storing both $\mathcal{A}$ ($M \times L$ matrix, considering

maximum hyperperiod to be L time units) and LUT ($q \times M$ matrix).

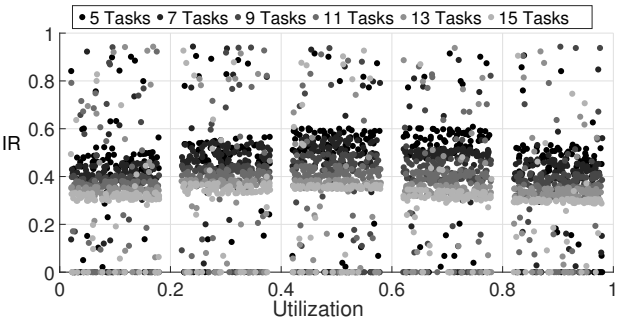
6 Experimental Evaluation

(1) Evaluation on synthetic task set: We generate synthetic task set evenly from each of the five utilization groups with utilization given by $[0.02 + 0.2i, 0.18 + 0.2i]$, $i = \{0, 1, 2, 3, 4\}$. Given a task set, its utilization means the sum of the utilization of all tasks in the set. In case of multi-rate tasks, only the smallest period is considered for maximum utilization. For example, tasks sets with utilization $\in [0.22, 0.38]$ are part of the utilization group with $i = 1$. In each of the five utilization groups, i.e., for $i = \{0, 1, 2, 3, 4\}$ we consider 6 tasks sets comprising (#trusted, #untrusted tasks) as (2,3), (3,4),(4,5),(6,7),(8,9) tasks. Thus, in total, we have 30 task sets (6 for each utilization group). Task sampling rates are chosen as divisors of 3000 (greater than 5) to establish a common maximum hyper-period of 3000 for all generated schedules. In this particular experiment, we consider tasks with lower periodicity to be of higher criticality and, hence, higher priority (which becomes the same as the rate monotonic case). For generating the synthetic task sets, we consider the utilization groups as discussed above, randomly set the task execution times within the $[1, 40]$ range and utilize algorithm *RandfixedSum* from [8] in order to generate task sets. The minimum periodicity of each task is also chosen by the *RandfixedSum* algorithm given the utilization groups. For every trusted task in the task set, at most two more secure sampling rates (that are divisors of 3000) are chosen using Claim 2 and Remark 1. The AEWs of trusted tasks are randomly chosen between $[10, 60]$ such that the AEW is sufficiently large (compared to the considered execution time range $[1, 40]$) to accommodate multiple tasks. We generate all possible fixed-priority preemptive schedules for the task sets using *TaskShuffler* and perform analyses of their IR and average AP by simulation. We have plotted IRs and average APs of 100 schedules randomly selected from the generated schedule set for each task set present within every utilization group. For each task set within a utilization group, the victim task τ_v and attacker task τ_a are randomly selected such that $pri(\tau_v) > pri(\tau_a)$, i.e. the victim task having higher priority than the attacker task. All the simulation experiments involving schedule generation and analysis are conducted using MATLAB and Python on an 8-core Intel i7 (7th generation) CPU with 16 GB of RAM.

(A) Inferability Ratio: We computed IR of 100 task schedules for every task set within the defined utilization range. IR is computed following Def. 1. The IR of schedules generated by the MAARS framework is illustrated as a scatter plot in Fig. 7a. Here, the x-axis represents the task utilization, while the y-axis represents the IR. The same experiment is repeated with schedules generated by attack-unaware randomization. Their IRs are plotted in Fig. 7b, where the x-axis and the y-axis represent the task utilisation and IR, respectively. In both these plots, the data points with *lighter* shades represent IR of schedules with *higher* number of tasks. Our results reveal that the MAARS framework achieves an average reduction of 37.4% in IR compared to the attack-unaware randomization approach (averaged over all schedules generated for a task set in a utilisation group). Additionally, we observe that schedules generated by the MAARS framework for task sets comprising a higher number of tasks show better improvement in terms of IR.



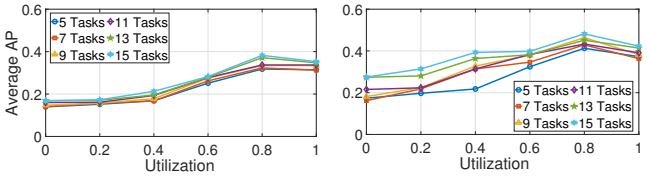
(a) IR: MAARS



(b) IR: Attack-unaware randomization

Figure 7: Comparison of Inferability Ratio

(B) Average Attack Probability: We computed the AP of 100 random schedules for each task set within the utilization ranges. The AP of each trusted task is determined using Def. 2. The average AP for $\tau_i \in \Gamma_t$ is denoted as $\bar{AP} = \frac{\sum_{j=1}^{100} AP_{\langle \tau_i, s_j \rangle}}{100}$ (averaged over 100 schedules) . These average APs of tasks are plotted in Fig. 8 by averaging over the schedules generated by the MAARS framework (in Fig. 8a) and attack unaware randomization (Fig. 8b). The x-axis and y-axis of both plots in Fig. 8 represent the task utilization and attack probability, respectively. We observe that average AP increases with utilization and number of tasks. With a higher number of tasks within a lower utilization range, MAARS performs significantly better than attack-unaware randomization. The schedules generated by the MAARS framework show a 62.7% average AP reduction compared to attack-unaware randomization.



(a) For MAARS Framework (b) For Attack-unaware randomization

Figure 8: Average AP: Varying the Number of Tasks and Utilization

(2). HIL evaluation of an automotive benchmark: We experimentally evaluate our proposed *MAARS* framework in a Hardware-in-Loop (HIL) testbed and compare it with state-of-the-art approaches against posterior SBAs. The first step of the experiment

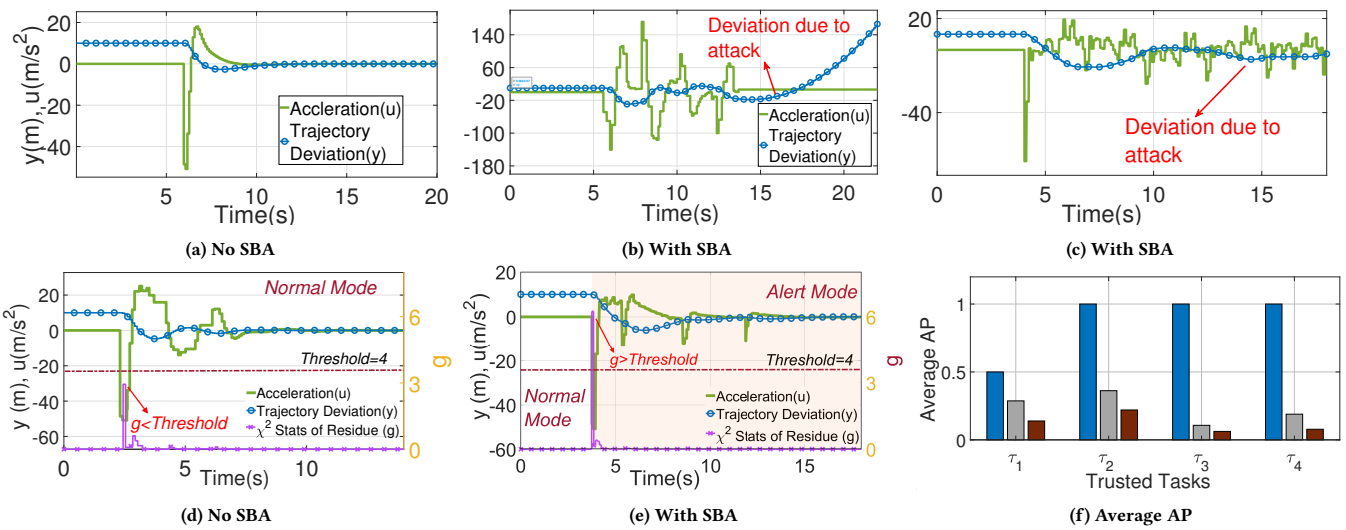


Figure 9: Plant Response with TTC: (a),(b) Using Static Priority Scheduler and (c) Using Attack-Unaware Randomization (d),(e) using MAARS Scheduling Framework (f) Average AP of control tasks from Table 3

involves *design-time* schedule generation and analysis for inferability and vulnerability of safety-critical trusted control tasks. In the next step, we perform a *run-time* analysis of the control tasks under posterior SBA. For this purpose, we employ an ARM-based 32-bit *Infineon Tricore Aurix-397 ECU*, connected to *ETAS Labcar*, a Real-time PC (RTPC), via Controller Area Network (CAN).

The physical system/plant is implemented on the RTPC, which sends the sensor measurement to the ECU via CAN, where the control tasks are co-scheduled. Following *AUTOSAR* mandates[9], we design separate reception tasks to filter and receive sensor IDs transmitted by the ETAS Labcar RTPC over CAN. After processing these sensor data, the control tasks (scheduled by the fixed-priority preemptive scheduler in the Infineon Aurix TC397 ECU) are executed. Subsequently, corresponding transmission tasks send computed control input through CAN to the RTPC, facilitating plant actuation. Our proposed run-time schedule selection process SchedSel() is scheduled to run once during each hyper-period in the same core as the other control tasks on the ECUs. Note that SchedSel() doesn't use data from the buffer; rather, it receives the detector signal in an interrupt-driven manner. The corresponding interrupt service routine provides a new schedule to the scheduler for the next hyperperiod such that the attacker cannot compromise task operations. We implement four distinct automotive controllers used in the work [11]. These control tasks include i) an electronic stability program (ESP) responsible for maintaining yaw stability, ii) a trajectory tracking control (TTC) system that regulates deviation from a desired longitudinal trajectory, iii) a cruise control (CC) system that ensures the desired vehicle speed, and iv) suspension control (SC) system that maintains vehicle suspension across varied roads and driving conditions. All four control tasks' design parameters, their respective AEWs, criticality levels, and TAPs are tabulated in the table Tab. 3. Apart from control tasks, 11 other tasks were implemented to facilitate reading from the data buffer, CAN Tx/Rx, message filtering, and implementing generic timers and interrupts. The timer resolution of the global timers in the ECU cores was set at $0.1ms$, i.e. time unit $\delta = 0.1ms$. The maximum hyperperiod length was set to $600ms$. The total number of data points stored in each hyperperiod is 6000.

Task	Control Loop	P_i (ms)	e_i(ms)	C_i	TAP	Ω_i	Schedules under TAP
τ_1	ESP	{5,12,15}	0.8	0.4	0.1	3	68
τ_2	TTC	{10,15,25}	2.3	0.3	0.2	4	762
τ_3	CC	{10,25}	1.6	0.2	0.1	1	1088
τ_4	SC	{20,25,30}	4.0	0.1	0.2	8	4023

Table 3: Trusted Task Set Parameters

(1) Design-time Analysis: To generate a set of security-aware task schedules, we synthesize a set of periodicities for the control tasks that ensure their control performance. To achieve this, we solve the LMIs given in Claim 1 using *YALMIP* [17] along with *Gurobi* solver. The performance preserving set of periodicities of each control task is further pruned following Claim 2 and Remark 1 to derive the secure sampling rates for all the control tasks. The final pruned set of security-aware performance preserving periodicities P_i of each control task τ_i is tabulated in column 3 of Tab. 3.

Next, we extend the *TaskShuffler* algorithm for multi-rate scheduling, as described in Sec. 5.2 to generate a set $\mathcal{S}$ of valid and randomly deployable schedules using P_i's. The total number of valid schedules generated is 14820. For each victim task in Tab. 3, we find the number of schedules for all victim tasks with attack probability under their respective TAPs (in column 8 of Tab 3). For each schedule in $\mathcal{S}$, we determine the attack probability of each trusted task using Def. 2. For each trusted task τ_i, we compute the attack probability AP_i^j considering the untrusted task τ_j is acting as an attacker. The average attack probability $\bar{AP}_i = \frac{\sum_{j=1}^{M} AP_{\langle \tau_i, s_j \rangle}}{M}$ is computed for each trusted task τ_i where M is the total number of schedules generated by MAARS. Fig. 9f plots the average AP values in the y-axis against each of the four trusted tasks (as given in Tab. 3) in the x-axis. The blue, grey, and brown bars in Fig. 9f represent the average APs

of schedules generated using the static priority scheduler, attack-unaware randomization, and the MAARS framework, respectively. We observed that the average AP is highest in the case of victim task τ_2, i.e. the TTC controller.

(2) Runtime Deployment in HIL: Following the analysis, we deployed the task setup in a Hardware-in-the-Loop (HIL) setup for evaluation. Given that τ_2, i.e. TTC exhibits the highest average attack probability, we designate it as the victim task for further evaluation. We consider settling time (t_s) and GUES decay rate (λ) metrics to evaluate the plant's performance. To launch the attack, a malicious code from one of the untrusted tasks is used that overrides the control input data of task τ_2, which is being sent to ETAS RTPC via CAN Tx task. A χ^2-detector is implemented along with τ_2 to detect any unnatural deviation in the plant states. Following the χ^2 statistics of the residue, the detector's threshold is set in order to 4 to maintain a false alarm rate (FAR) $\leq 2\%$. We compare the performance of the TTC system under posterior SBA in Fig. 9 when schedules generated by the MAARS framework, attack-unaware randomization and static fixed priority schedule are deployed. In Fig. 9, the green, blue, red, and magenta plots respectively represent the control input acceleration, plant state deviation from the reference trajectory, the detector's threshold, and χ^2 statistics of the detector. Under no SBA, we note $t_s = 4.3s$ (<desired $t_s = 10$), and $\lambda = 95\%$ when a fixed priority schedule is used in Fig. 9a. However, we can observe in Fig. 9b that the plant becomes unstable due to posterior SBA under the static fixed priority schedule. Figure 9c shows the plant response under attack with attack-unaware randomization. In this case, we noted $t_s = 21s$ and $\lambda = 76\%$. On the other hand, Fig. 9d shows the behaviour of the TTC system under no attack when MAARS is deployed. We can observe $t_s = 6s$ and $\lambda = 93\%$. Under posterior SBA, the scheduler switches from normal mode to alert mode once χ^2 residue> 4 (red highlighted area in Fig. 9e). In alert mode, we note $t_s = 9.2s$ and $\lambda = 89\%$. This implies that the system is resilient against posterior attack when MAARS is deployed, and subsequently, the performance of the system is preserved. The runtime overhead of the MAARS framework is 0.0344ms in normal mode and 0.0763ms in alert mode, indicating that the MAARS framework is efficient and incurs minimal performance overhead, making it suitable for real-time systems. The memory required to store the schedules is 3.22 Megabytes, which is well within the 16 Megabyte flash capacity of the Infineon TC397 ECU used in our experiment.

7 Conclusion

Existing schedule randomization methodologies for securing real-time task schedules are attack-unaware and susceptible to schedule-based attacks. Also, they are not suitable for control-theoretic tasks in general. In this paper, we propose an attack-aware framework that deploys suitable task sampling rates, which reduce schedule vulnerability against posterior attacks. The method is aware of control performance issues in case the task under question is a control task. Our results show that the MAARS framework effectively reduces the inferability of critical trusted task parameters while also reducing posterior attack probability. In the future, we aim to extend this performance-aware security framework with support for multi-processor systems, which have become prevalent in automotive control and other CPS domains.

References

[1] 2024. OSEK Specification 2.2.3. https://www.osek-vdx.org/mirror/os223.pdf.

[2] Sunandan Adhikary et al. 2022. Work-in-Progress: Control Skipping Sequence Synthesis to Counter Schedule-based Attacks. In *RTSS*. IEEE.

[3] Sunandan Adhikary, Ipsita Koley, Saurav Kumar Ghosh, Sumana Ghosh, Soumyajit Dey, and Debdeep Mukhopadhyay. 2020. Skip to secure: Securing cyber-physical control loops with intentionally skipped executions. In *Proceedings of the 2020 Joint Workshop on CPS&IoT Security and Privacy*.

[4] Sanjoy K Baruah, Alan Burns, and Robert I Davis. 2011. Response-time analysis for mixed criticality systems. In *2011 IEEE 32nd Real-Time Systems Symposium*.

[5] Chien-Ying Chen, Sibin Mohan, Rodolfo Pellizzoni, Rakesh B Bobba, and Negar Kiyavash. 2019. A novel side-channel in real-time schedulers. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE.

[6] Chien-Ying Chen, Debopam Sanyal, and Sibin Mohan. 2021. Indistinguishability prevents scheduler side channels in real-time systems. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*.

[7] Jiyang Chen, Tomasz Kloda, Rohan Tabish, Ayoosh Bansal, Chien-Ying Chen, Bo Liu, Sibin Mohan, Marco Caccamo, and Lui Sha. 2022. SchedGuard++: Protecting against Schedule Leaks Using Linux Containers On Multi-Core Processors. *ACM Trans. Cyber-Phys. Syst.* (2022). doi:10.1145/3565974

[8] Paul Emberson, Roger Stafford, and Robert I Davis. 2010. Techniques for the synthesis of multiprocessor tasksets. In *proceedings 1st International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS 2010)*.

[9] Simon Fürst, Jürgen Mössinger, Stefan Bunzel, Thomas Weber, Frank Kirschke-Biller, Peter Heitkämper, Gerulf Kinkelin, Kenji Nishikawa, and Klaus Lange. 2009. AUTOSAR–A Worldwide Standard is on the Road. In *14th International VDI Congress Electronic Systems for Vehicles, Baden-Baden*. Citeseer.

[10] Dan Henriksson and Anton Cervin. 2005. Optimal on-line sampling period assignment for real-time control tasks based on plant state information. In *Proceedings of the 44th IEEE Conference on Decision and Control*. IEEE.

[11] Ipsita Koley, Sunandan Adhikary, and Soumyajit Dey. 2021. Catch me if you learn: real-time attack detection and mitigation in learning enabled CPS. In *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE.

[12] Ipsita Koley, Soumyajit Dey, Debdeep Mukhopadhyay, Sachin Singh, Lavanya Lokesh, and Shantaram Vishwanath Ghotgalkar. 2023. CAD Support for Security and Robustness Analysis of Safety-critical Automotive Software. *ACM Transactions on Cyber-Physical Systems* (2023).

[13] Kristin Krüger, Marcus Volp, and Gerhard Fohler. 2018. Vulnerability analysis and mitigation of directed timing inference based attacks on time-triggered systems. *LIPIcs-Leibniz International Proceedings in Informatics* (2018).

[14] Kristin Krüger, Nils Vreman, Richard Pates, Martina Maggio, Marcus Volp, and Gerhard Fohler. 2021. Randomization as Mitigation of Directed Timing Inference Based Attacks on Time-Triggered Real-Time Systems with Task Replication. *LITES: Leibnitz Transactions on Embedded Systems* (2021).

[15] Jaewoo Lee and Jinkyu Lee. 2021. MC-FLEX: Flexible Mixed-Criticality Real-Time Scheduling by Task-Level Mode Switch. *IEEE Trans. Comput.* (2021).

[16] Daniel Liberzon and A Stephen Morse. 1999. Basic problems in stability and design of switched systems. *IEEE control systems magazine* (1999).

[17] Johan Lofberg. 2004. YALMIP: A toolbox for modeling and optimization in MATLAB. In *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE.

[18] Mitra Nasri, Thidapat Chantem, Gedare Bloom, and Ryan M Gerdes. 2019. On the pitfalls and vulnerabilities of schedule randomization against schedule-based attacks. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE.

[19] Nelson Prates, Andressa Vergütz, Ricardo T Macedo, Aldri Santos, and Michele Nogueira. 2020. A defense mechanism for timing-based side-channel attacks on IoT traffic. In *GLOBECOM 2020 IEEE Global Communications Conference*.

[20] Jiankang Ren, Chunxiao Liu, Chi Lin, Ran Bi, Simeng Li, Zheng Wang, Yicheng Qian, Zhichao Zhao, and Guozhen Tan. 2023. Protection Window Based Security-Aware Scheduling against Schedule-Based Attacks. *ACM Transactions on Embedded Computing Systems* (2023).

[21] Arkaprava Sain, Suraj Singh, Sunandan Adhikary, Ipsita Koley, and Soumyajit Dey. 2023. Work-in-Progress: Securing Safety-Critical Control Tasks with Attack-aware Multi-Rate Scheduling. In *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE.

[22] Michael Schinkel et al. 2002. Optimal control for systems with varying sampling rate. In *ACC*. IEEE.

[23] Man-Ki Yoon, Sibin Mohan, Chien-Ying Chen, and Lui Sha. 2016. Taskshuffler: A schedule randomization protocol for obfuscation against timing inference attacks in real-time systems. In *2016 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE.

A Proof of Claim 2

Proof. We consider that an attacker constructs the schedule ladder with a row length equal to the minimum sampling period p_i of the victim task $\tau_i \in \Gamma_t$ and observes the AAI and AEI of an attacker task $\tau_j \in \Gamma_u$. The duration between the attacker task's arrival and the subsequent victim task's arrival repeats after every $LCM(p_i, p_j)$ unit of time. Note that e_j time units are covered by every execution instance of the attacker task. Note that, $p_i, p_j, e_i, e_j \in \mathbb{N}^+$ because each represents a countable number of time units, measured in integer multiples of the smallest time unit δ in the ladder construction. As long as the duration between the attacker's and the subsequent victim's arrival is smaller than e_j time units, the attacker task will be preempted by the victim task, and the same preemption will repeat after every multiple of $LCM(p_i, p_j)$ in the timeline. Now, let us consider that at k-th instance of the victim task, there is a predicted chance of preemption while the victim is running with some np_i periodicity where $n, k \in \mathbb{N}^+$. Therefore, this k-th victim instance that arrives at knp_i time may preempt the $\lfloor \frac{knp_i}{p_j} \rfloor$-th instance of the attacker task. We examine two scenarios to illustrate the preemptions of the attacker task: (1) When a low-priority attacker task is executing, and a high-priority victim task arrives, the attacker task will be preempted. (2) When a high-priority victim task is executing, and a lower-priority attacker task arrives, the attacker task is preempted. We analyse each case separately.

Case 1: For successful preemption, the k-th victim task instance must *arrive before the execution of* $\lfloor \frac{knp_i}{p_j} \rfloor$*-th attacker instance is finished, i.e.,* $\lfloor \frac{knp_i}{p_j} \rfloor p_j + e_j > knp_i$ Now, consider changing the periodicity of the victim task to some p_i', to avoid this preemption. Therefore p_i' must ensure that the k-th victim instance arrives after the $\lfloor \frac{kp_i'}{p_j} \rfloor$-th attacker instance finishes, i.e., $\lfloor \frac{kp_i'}{p_j} \rfloor p_j + e_j \leq kp_i'$. By substituting p_i' with $np_i + e_j$, this equation becomes as follows.

$$
\begin{aligned}
k(np_i + e_j) &\geq \lfloor \frac{k(np_i + e_j)}{p_j} \rfloor p_j + e_j = \lfloor \frac{k(np_i)}{p_j} + \frac{k(e_j)}{p_j} \rfloor p_j + e_j \\
&\geq \lfloor \frac{k(np_i)}{p_j} + \lfloor \frac{ke_j}{p_j} \rfloor + ke_j \% p_j \rfloor p_j + e_j, \\
&\qquad\qquad \text{since } \frac{a}{b} = \lfloor \frac{a}{b} \rfloor + a\%b \\
&\geq \lfloor \frac{k(np_i)}{p_j} \rfloor p_j + ke_j + ke_j \% p_j + e_j, \text{ since } k, e_j, p_j \in \mathbb{N}^+ \\
&\geq knp_i - knp_i \% p_j + ke_j - ke_j \% p_j \\
&\geq k(np_i + e_j) - k(np_i \% p_j - e_j \% p_j)
\end{aligned}
$$

This always holds true since $k(np_i\%p_j - e_j\%p_j) > 0$, because for τ_j to be schedulable $p_j > e_j$. Note that $e_j,\ p_j$ are considered whole numbers since we express them in multiples of δ, and the analysis assumes δ as the notion of unit time. Therefore, this choice of periodicity $p_i' = np_i + e_j$ ensures no preemption of the attacker by any k-th victim instance, ensuring least possible IR, hence proving Claim 2.

Case 2: To get preempted by the k-th victim task instance, the $\lceil \frac{knp_i}{p_j} \rceil$-th instance of the attacker task must arrive before k-th victim task have finished it's execution, i.e. $knp_i + e_i > \lceil \frac{knp_i}{p_j} \rceil p_j$. Now, we switch the periodicity of the victim task to some p_i' to avoid this preemption. Therefore p_i' must ensure that the $\lceil \frac{knp_i'}{p_j} \rceil$-th attacker task arrives after k-th victim task has finished executing. Therefore, by substituting p_i' with $np_i + e_j$, the equation becomes:

$$
\begin{aligned}
k(np_i + e_j) + e_i &\leq \lceil \frac{k(np_i + e_j)}{p_j} \rceil p_j \\
&\leq \left\lfloor \frac{k(np_i + e_j) + p_j - (knp_i + e_j)\%p_j}{p_j} \right\rfloor p_j, \\
&\qquad \text{since } \lceil \frac{a}{b} \rceil = \frac{a}{b} + \frac{b - (a\%b)}{b} \\
&\leq k(np_i + e_j) + p_j - (knp_i + e_j)\%p_j
\end{aligned}
$$

After cancelling $k(np_i + e_j)$ in both sides of the inequality, we get: $e_i \leq p_j - \left[(knp_i + e_j)\%p_j\right]$. The term $(knp_i + e_j)\%p_j$ is remainder when $(knp_i + e_j)$ is divided by p_j. The maximum possible value of this remainder is $p_j - 1$ (and minimum value 0 naturally). Therefore, $e_i \in [0, p_j)$. Note that since $pri(\tau_j) < pri(\tau_i)$, it is highly unlikely that a critical task τ_i will have a higher execution time than the periodicity of a low-criticality untrusted task τ_j. This proves that the claim that *scheduling a high-criticality victim task* $\tau_i \in \Gamma_t$ *with a set of periodicities* $P_i = \{p_i' | p_i' = np_i + e_j\}$ *ensures the least IR for any attacker task* $\tau_j \in \Gamma_u$ *in an execution schedule for most practical scenarios.* □